LexCorp Law Firm Management Portal: Engineering Proposal

Type: Solo lab project (database + web app) | **Architecture:** Monolith, server-side rendered | **Hard constraints:** No ORM, no API framework, raw parameterized SQL only

---

1. Project Summary

LexCorp is an internal management portal for a corporate law firm. Staff log in, manage the staff directory, view matters (cases), log billable hours, see partner revenue rankings, check new clients for conflicts of interest, and monitor overdue invoices.

The project must demonstrate **six distinct SQL concepts** through six working features, plus authentication, validation and security basics. Every feature below is mapped to one SQL concept so each earns its own marks.

#	Feature	Primary SQL concept
1	Staff Directory	Self-join (LEFT JOIN) + INSERT
2	Active Matters Dashboard	3-way INNER JOIN + filtering
3	Billable Hours Tracker	INSERT + aggregation readback + calculated fields
4	Rainmaker Leaderboard	JOIN + GROUP BY + SUM + ORDER BY + LIMIT
5	Conflict of Interest Checker	Subqueries (EXISTS / IN)
6	Overdue Invoice Alerts	Date filtering + GROUP BY + HAVING

---

2. Tech Stack (Fixed)

Layer	Choice	Notes
Database	MySQL 8 (PostgreSQL also fine)	MySQL 8 enforces CHECK constraints
Backend	Python 3.11+ with Flask	Flask is a web server, not an ORM, so it satisfies the rules
DB driver	`mysql-connector-python`	Raw `cursor.execute(sql, params)` only

Layer	Choice	Notes
Templates	Jinja2 (bundled with Flask)	Auto-escapes output, which prevents XSS
Frontend	HTML5, CSS3, vanilla JS, Bootstrap 5 via CDN	No frameworks
Auth	`werkzeug.security` + Flask `session`	Password hashing, no plain text

Rule for the whole codebase: no string-formatted SQL. Every query uses `%s` placeholders with a parameter tuple. No exceptions.

---

3. Repository Structure

```
lexcorp/
├── app.py                # Flask app, routes
├── db.py                 # get_connection(), helper functions
├── config.py             # DB credentials, SECRET_KEY (env vars)
├── schema.sql            # CREATE TABLE statements
├── seed.sql              # Dummy data
├── requirements.txt
├── templates/
│   ├── base.html         # Layout, navbar, flash messages
│   ├── login.html
│   ├── staff.html
│   ├── matters.html
│   ├── time_entry.html
│   ├── leaderboard.html
│   ├── conflict.html
│   └── overdue.html
└── static/
    ├── css/theme.css     # CSS variables, navy/gold theme
    └── js/validate.js    # Client-side validation
```

---

4. Database Specification

4.1 ER Overview

- A **Client** has many **Matters**.

- A **Matter** has one lead **Staff** member (a Partner), many **TimeEntries**, and many **Invoices**.
- **Staff** references itself via `manager_id` (hierarchy).
- A **TimeEntry** belongs to one Staff member and one Matter.

Deliverable: draw this as an ER diagram (draw.io or dbdiagram.io) and export to PNG for the report.

4.2 Schema (`schema.sql`)

```
CREATE TABLE Staff (
    staff_id       INT AUTO_INCREMENT PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    email          VARCHAR(120) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,
    role           ENUM('Managing
Partner','Partner','Associate','Paralegal','Admin') NOT NULL,
    practice_area  VARCHAR(60) NOT NULL,
    hourly_rate    DECIMAL(8,2) NOT NULL CHECK (hourly_rate >= 0),
    manager_id     INT NULL,
    hire_date      DATE NOT NULL,
    FOREIGN KEY (manager_id) REFERENCES Staff(staff_id) ON DELETE SET NULL
);

CREATE TABLE Clients (
    client_id      INT AUTO_INCREMENT PRIMARY KEY,
    company_name   VARCHAR(150) NOT NULL UNIQUE,
    industry       VARCHAR(80) NOT NULL,
    contact_email  VARCHAR(120),
    created_at     DATE NOT NULL DEFAULT (CURRENT_DATE)
);

CREATE TABLE Matters (
    matter_id        INT AUTO_INCREMENT PRIMARY KEY,
    matter_title     VARCHAR(200) NOT NULL,
    client_id        INT NOT NULL,
    lead_staff_id    INT NOT NULL,
    opposing_party   VARCHAR(150) NULL,
    status           ENUM('Active','On Hold','Closed') NOT NULL DEFAULT
'Active',
    opened_date      DATE NOT NULL,
    FOREIGN KEY (client_id)     REFERENCES Clients(client_id) ON DELETE
RESTRICT,
    FOREIGN KEY (lead_staff_id) REFERENCES Staff(staff_id)    ON DELETE
RESTRICT
```

```
);

CREATE TABLE TimeEntries (
    entry_id       INT AUTO_INCREMENT PRIMARY KEY,
    staff_id       INT NOT NULL,
    matter_id      INT NOT NULL,
    hours_logged   DECIMAL(5,2) NOT NULL CHECK (hours_logged > 0 AND
hours_logged <= 24),
    entry_date     DATE NOT NULL DEFAULT (CURRENT_DATE),
    description    VARCHAR(500) NOT NULL,
    FOREIGN KEY (staff_id)  REFERENCES Staff(staff_id)    ON DELETE RESTRICT,
    FOREIGN KEY (matter_id) REFERENCES Matters(matter_id) ON DELETE RESTRICT
);

CREATE TABLE Invoices (
    invoice_id     INT AUTO_INCREMENT PRIMARY KEY,
    matter_id      INT NOT NULL,
    amount         DECIMAL(12,2) NOT NULL CHECK (amount > 0),
    issue_date     DATE NOT NULL,
    due_date       DATE NOT NULL,
    status         ENUM('Paid','Unpaid','Void') NOT NULL DEFAULT 'Unpaid',
    FOREIGN KEY (matter_id) REFERENCES Matters(matter_id) ON DELETE RESTRICT,
    CHECK (due_date >= issue_date)
);

CREATE INDEX idx_matters_status   ON Matters(status);
CREATE INDEX idx_invoices_status  ON Invoices(status, due_date);
CREATE INDEX idx_clients_industry ON Clients(industry);
CREATE INDEX idx_time_matter      ON TimeEntries(matter_id);
```

4.3 Delete-Rule Decisions (Viva Talking Point)

Relationship	Rule	Reason
Staff.manager_id to Staff	`SET NULL`	If a manager leaves, their reports remain; reassign later
Matters to Clients/Staff	`RESTRICT`	Legal records must not vanish when a client or lawyer is removed
TimeEntries, Invoices to Matters	`RESTRICT`	Financial and billing records need an audit trail

Do **not** use `ON DELETE CASCADE` on financial tables. Be ready to explain why.

4.4 Invoice and Time Entry Relationship

Invoices are **seeded** for the core build. They are not computed from time entries. State this explicitly in the report. Optional stretch (Section 8) generates an invoice from time entries inside a transaction.

4.5 Seed Data Requirements (`seed.sql`)

Table	Minimum rows	Notes
Staff	12	1 Managing Partner, 3 to 4 Partners, 4 Associates, 2 Paralegals, 1 Admin; at least 3 practice areas; hierarchy 3 levels deep
Clients	10	At least 3 industries with 2+ clients each (needed for the conflict demo)
Matters	15	Mix of Active, On Hold, Closed; at least 3 with `opposing_party` set
TimeEntries	40	Spread across staff, matters and the last 90 days
Invoices	20	About 8 Paid, 8 Unpaid, 4 Void; at least 4 overdue; at least 2 clients with overdue total above 10,000

Tip: use relative dates in seed data so overdue items stay overdue when you demo:

```
-- overdue by 20 days
INSERT INTO Invoices (matter_id, amount, issue_date, due_date, status)
VALUES (3, 18500.00, DATE_SUB(CURDATE(), INTERVAL 50 DAY),
DATE_SUB(CURDATE(), INTERVAL 20 DAY), 'Unpaid');
```

Create the seeded staff passwords with `generate_password_hash()` in a small one-off Python script, then paste the hashes into `seed.sql`. Never store plain text.

---

5. Backend Specification

5.1 Connection (`db.py`)

- One function `get_connection()` that reads credentials from environment variables.
- Open a connection per request and close it in a `finally` block (or use Flask's `teardown_appcontext`).
- Use `cursor(dictionary=True)` so templates can use `row.name`.

- Wrap every write in `try / except`, call `conn.rollback()` on failure, `conn.commit()` on success.

5.2 Security Checklist (all mandatory)

- [ ] Every query is parameterized with `%s`.
- [ ] Passwords hashed with `generate_password_hash`, checked with `check_password_hash`.
- [ ] `SECRET_KEY` loaded from an environment variable, not hardcoded.
- [ ] Login required on every route except `/login` (use a `@login_required` decorator).
- [ ] Server-side validation on every POST (client-side JS is a convenience only).
- [ ] Jinja auto-escaping left on; never use `|safe` on user input.
- [ ] Dynamic filter values (practice area) passed as parameters. Never concatenate them into SQL.
- [ ] Generic error pages; do not leak SQL errors to the browser.
- [ ] Role check: only Partners and the Managing Partner can view Leaderboard and Overdue pages (small `role` check in the decorator).

5.3 Route Map

Method	Route	Purpose
GET, POST	`/login`	Authenticate, create session
GET	`/logout`	Clear session
GET	`/`	Dashboard home with links and summary counts
GET	`/staff`	Staff directory (optional `?practice_area=`)
POST	`/staff/add`	Insert new staff member
GET	`/matters`	Active matters table
GET, POST	`/time`	Log hours form, recent entries, per-matter totals
GET	`/leaderboard`	Top 5 partners by paid revenue
GET, POST	`/conflict`	Conflict checker form and result
GET	`/overdue`	Overdue invoice dashboard

---

6. Feature Specifications

Authentication (cross-cutting, build first after the schema)

- **Frontend:** login form (email, password), Bootstrap card, error flash on failure.
- **Backend:** `SELECT staff_id, name, role, password_hash FROM Staff WHERE email = %s;` then `check_password_hash`. On success store `staff_id`, `name`, `role` in `session`.
- **Acceptance:** wrong password shows a generic error; unauthenticated access redirects to `/login`.

Feature 1: Staff Directory (Self-Join and Insert)

Frontend

- Table: Name, Role, Practice Area, Hourly Rate, Manager.
- Dropdown filter populated from `SELECT DISTINCT practice_area FROM Staff ORDER BY practice_area;`.
- Add-staff form: name, email, temporary password, role, practice area, hourly rate, manager (dropdown of existing staff, optional), hire date.

Backend queries

```
-- List with optional filter (pass the same value twice; None means no
filter)
SELECT e.staff_id, e.name, e.role, e.practice_area, e.hourly_rate,
       m.name AS manager
FROM Staff e
LEFT JOIN Staff m ON e.manager_id = m.staff_id
WHERE (%s IS NULL OR e.practice_area = %s)
ORDER BY e.name;

-- Insert
INSERT INTO Staff (name, email, password_hash, role, practice_area,
hourly_rate, manager_id, hire_date)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s);
```

Validation: name and email required; email format and uniqueness (catch the duplicate-key error and show a friendly message); rate is a number of at least 0; role must be one of the enum values; a person cannot be their own manager.

Acceptance: adding a staff member with a manager shows that manager's name in the table; filtering by practice area narrows the list; employees without managers still appear (this is why it is a LEFT JOIN).

Feature 2: Active Matters Dashboard (3-Way Join)

Frontend: table with Matter Title, Client, Lead Partner, Opened Date, Opposing Party. Optional dropdown to filter by status (Active, On Hold, Closed).

Backend query

```
SELECT m.matter_id, m.matter_title, c.company_name, s.name AS lead_partner,
       m.opened_date, m.opposing_party
FROM Matters m
JOIN Clients c ON m.client_id = c.client_id
JOIN Staff   s ON m.lead_staff_id = s.staff_id
WHERE m.status = %s
ORDER BY m.opened_date DESC;
```

Acceptance: default view shows only Active matters; changing the filter changes results; every row shows both client and lead partner names, not IDs.

Feature 3: Billable Hours Tracker (Write Path and Readback)

Frontend

- Form: Matter (dropdown of Active matters), Hours (number, 0.25 step), Date (defaults to today), Description (required textarea).
- Below the form: a table of the logged-in user's last 10 entries, and a per-matter summary table.

Backend queries

```
-- Insert (staff_id comes from the session, never from the form)
INSERT INTO TimeEntries (staff_id, matter_id, hours_logged, entry_date,
description)
VALUES (%s, %s, %s, %s, %s);

-- Per-matter summary with billable amount (join + aggregation + calculation)
SELECT m.matter_title,
       SUM(t.hours_logged)                  AS total_hours,
       SUM(t.hours_logged * s.hourly_rate)   AS billable_amount
FROM TimeEntries t
JOIN Matters m ON t.matter_id = m.matter_id
JOIN Staff   s ON t.staff_id  = s.staff_id
GROUP BY m.matter_id, m.matter_title
ORDER BY billable_amount DESC;
```

Server-side validation: matter exists and is Active; hours greater than 0 and at most 24; date is not in the future; description 1 to 500 characters. Reject and re-render the form with errors if any check fails.

Acceptance: submitting a valid entry immediately updates the summary table; submitting 0 or 25 hours is rejected even if JS is disabled.

Feature 4: Rainmaker Leaderboard (Aggregation)

Frontend: ranked table (rank, partner name, practice area, total paid revenue formatted as currency). Optionally a Bootstrap progress bar showing each partner's share of the top earner.

Backend query

```
SELECT s.name, s.practice_area, SUM(i.amount) AS revenue
FROM Staff s
JOIN Matters  m ON s.staff_id = m.lead_staff_id
JOIN Invoices i ON m.matter_id = i.matter_id
WHERE i.status = 'Paid'
GROUP BY s.staff_id, s.name, s.practice_area
ORDER BY revenue DESC
LIMIT 5;
```

Acceptance: Unpaid and Void invoices never count; results are sorted descending; changing an invoice from Paid to Unpaid in the DB changes the ranking.

Feature 5: Conflict of Interest Checker (Subqueries)

A real conflict of interest means the firm would act for, or against, the same party in a way that creates divided loyalties. Model it with two levels:

- **Direct conflict (red alert):** the prospective client is currently an *opposing party* in an active matter.
- **Industry overlap (yellow warning):** the firm already represents active clients in the same industry; flag for partner review.

Frontend: form with Prospective Client Name and Industry. Result panel shows a red Bootstrap alert for a direct conflict (naming the matter), a yellow alert listing overlapping clients, or a green "No conflicts found".

Backend queries

```
-- Direct conflict: EXISTS subquery (returns 1 or 0)
SELECT EXISTS (
    SELECT 1 FROM Matters
    WHERE status = 'Active'
      AND LOWER(opposing_party) = LOWER(%s)
) AS direct_conflict;

-- Which matters? (for the alert text)
SELECT matter_title, opposing_party FROM Matters
WHERE status = 'Active' AND LOWER(opposing_party) = LOWER(%s);

-- Industry overlap: IN subquery
SELECT company_name FROM Clients
WHERE industry = %s
  AND client_id IN (SELECT client_id FROM Matters WHERE status = 'Active');
```

Acceptance: typing the exact name of a seeded opposing party triggers the red alert (case-insensitive); an industry with active clients triggers the yellow warning; an unknown name and industry gives green.

Feature 6: Overdue Invoice Alerts (Date Logic and HAVING)

Frontend: red-themed dashboard. Top banner: total overdue amount and number of clients. Table: Client, Number of Overdue Invoices, Total Overdue, Oldest Due Date, Days Overdue.

Backend query

```
SELECT c.company_name,
       COUNT(i.invoice_id)                      AS overdue_invoices,
       SUM(i.amount)                            AS total_overdue,
       MIN(i.due_date)                          AS oldest_due_date,
       DATEDIFF(CURDATE(), MIN(i.due_date))     AS days_overdue
FROM Clients c
JOIN Matters  m ON c.client_id = m.client_id
JOIN Invoices i ON m.matter_id = i.matter_id
WHERE i.status = 'Unpaid'
  AND i.due_date < CURDATE()
GROUP BY c.client_id, c.company_name
HAVING SUM(i.amount) > 10000
ORDER BY total_overdue DESC;
```

Acceptance: a client with two overdue invoices of 6,000 each appears (total 12,000) even though neither alone exceeds 10,000; Paid and Void invoices are excluded; future-dated unpaid invoices are excluded.

7. Frontend Specification

Theme: dark navy and black backgrounds, gold accents, serif headings. Define everything as CSS variables so the theme is one file:

```
:root {
  --bg: #0b1020;
  --surface: #141b2e;
  --gold: #c9a227;
  --text: #e8e6e1;
  --danger: #b3261e;
  --heading-font: 'Playfair Display', Georgia, serif;
  --body-font: system-ui, sans-serif;
}
```

Rules

- Bootstrap 5 via CDN for grid, tables, forms, alerts, navbar; override colors in `theme.css`.
- Playfair Display for headings via Google Fonts, with Georgia fallback.
- Tables use `table-responsive` wrappers; currency formatted with 2 decimals and thousands separators.
- No animations beyond Bootstrap defaults.
- `validate.js`: required fields, numeric ranges, email pattern, disabling the submit button on click to prevent double posts.
- Flash messages (success, error) rendered in `base.html`.
- Empty states: if a query returns zero rows, show "No records found", not a blank table.

---

8. Optional Stretch Goals (Only After All Six Features Work)

1. **Generate Invoice (transaction demo):** on a matter page, sum unbilled time entries times staff rate, insert an Invoice, and mark those entries as billed, all inside one transaction with `rollback` on error. This also resolves the "how do invoices relate to time" question. It needs a `billed BOOLEAN DEFAULT FALSE` column on TimeEntries.
2. **SQL View:** `CREATE VIEW v_active_matters AS ...` using the Feature 2 query; select from the view in the route.
3. **Stored procedure or trigger:** for example, a trigger that blocks time entries on Closed matters.

4. **Edit and delete** for staff and clients, to show full CRUD (UPDATE and DELETE with RESTRICT errors handled gracefully).

---

9. Build Order and Timeline

Phase	Tasks	Done when
1. Foundation	ER diagram, `schema.sql`, `seed.sql`, run both cleanly on a fresh DB	`SELECT COUNT(*)` on every table returns the expected rows
2. Skeleton	Flask app, `db.py`, `base.html`, theme, login and logout, `@login_required`	You can log in as a seeded user and see a blank dashboard
3. Quick win	Feature 2 (read-only)	Matters table renders with real data
4. Write path	Feature 3 with validation	Insert works and summary updates
5. Directory	Feature 1	Filter and add both work
6. Analytics	Features 4, 6, then 5	All three pass acceptance tests
7. Hardening	Role checks, error pages, empty states, security checklist review	Every checkbox in Section 5.2 is ticked
8. Report and viva prep	Screenshots, ER diagram, query explanations, rehearsal	You can explain every query without notes

---

10. Rubric Mapping

Before starting, open your assignment brief and copy each rubric line into this table. If the rubric mentions something not listed here (transactions, views, indexes), add it to the plan.

Rubric item (fill in exact wording)	Where it is satisfied
Database design, PK/FK, constraints	`schema.sql`, ER diagram
Raw SQL, no ORM	All routes in `app.py`
Joins	Features 1, 2, 4, 6

Rubric item (fill in exact wording)	Where it is satisfied
Aggregation, GROUP BY, HAVING	Features 3, 4, 6
Subqueries	Feature 5
CRUD and live connectivity	Features 1 and 3
Security (SQL injection, hashing, validation)	Section 5.2
UI and usability	Section 7

---

11. Testing Checklist

Functional

- [ ] Each feature's acceptance criteria pass with seeded data.
- [ ] Each page renders correctly with zero rows (temporarily empty a table).

Security

- [ ] Enter `' OR '1'='1` in the login email and every text field; the app behaves normally.
- [ ] Enter `<script>alert(1)</script>` as a client or matter description; it displays as text, not code.
- [ ] Visit `/staff` while logged out; you are redirected to `/login`.
- [ ] Log in as an Associate and try `/leaderboard` directly; access is denied.
- [ ] Disable JavaScript and submit invalid hours; the server rejects them.

Data integrity

- [ ] Try to delete a client who has matters; it is blocked by `RESTRICT`.
- [ ] Try to insert a negative rate or an invoice due before it was issued; the CHECK constraint rejects it.

---

12. Viva Preparation: Questions You Should Be Able to Answer

1. **Why a LEFT JOIN in Feature 1?** Top-level staff have no manager; an inner join would drop them.
2. **Why RESTRICT instead of CASCADE on financial tables?** Legal and financial records need an audit trail and must not disappear through an accidental delete.
3. **How do you prevent SQL injection?** Parameterized queries; the driver sends values separately from the SQL text.

4. **Why `GROUP BY s.staff_id, s.name`?** Every non-aggregated selected column should appear in the GROUP BY for strict SQL modes and portability.
5. **WHERE versus HAVING?** WHERE filters rows before grouping; HAVING filters groups after aggregation (Feature 6).
6. **Why does Feature 5 use a subquery?** The conflict test depends on data from another table (Matters), and `EXISTS` stops at the first match, so it is efficient.
7. **Why validate on the server as well as the browser?** Client-side checks can be bypassed; the server is the only trusted layer.
8. **Why did you add indexes?** To speed lookups and joins on frequently filtered columns (`status`, `due_date`, `industry`).
9. **How are invoices related to time entries?** In the core build they are seeded; in the stretch goal they are generated from time entries in a transaction.
10. **What is the real-world definition of a conflict of interest, and how does your model reflect it?** Direct conflict means acting against a current client; the checker detects opposing-party matches and flags industry overlap for review.

---

13. Definition of Done

- [ ] `schema.sql` and `seed.sql` run cleanly from scratch.
- [ ] All six features work end to end with the acceptance criteria met.
- [ ] Login, sessions, role checks and hashed passwords work.
- [ ] No SQL string formatting anywhere in the codebase (search for `f"` and `%` near `execute`).
- [ ] README with setup steps: create DB, run schema and seed, set environment variables, `pip install -r requirements.txt`, `flask run`.
- [ ] ER diagram, screenshots of each feature, and the list of queries are ready for the report.
- [ ] You have rehearsed the Section 12 questions out loud.